GUIDE 02

Frontend Environment Setup in Google Antigravity

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Install and configure Google Antigravity IDE for a React workspace.
- Create a React + TypeScript SPA using Vite.
- Apply a scalable feature-first folder structure.
- Configure Redux Toolkit, Material UI, Tailwind CSS and routing without style conflicts.

Prerequisites

- Windows 10/11 64-bit, macOS, or a supported Linux distribution.
- Node.js 22.12 or later and npm.
- A GitHub account for later labs.

Suggested student time: 2.5-3 hours

Responsible use: Google Antigravity is an AI-assisted IDE. Students remain responsible for reading generated diffs, running tests, explaining code, and avoiding insertion of passwords, tokens, student data, or assessment answers into prompts.

1. Install Google Antigravity IDE

1. Open the official Antigravity download page and choose the installer for your operating system.
2. Run the installer. On Windows, use a 64-bit supported edition; on Linux, confirm the distribution meets the documented glibc requirements.
3. Launch Antigravity and sign in only with an account permitted by your institution.
4. Open Settings and enable format-on-save. Install or enable ESLint, Prettier, Git integration, and the official language support already bundled with the IDE.
5. Create a parent folder named UniMart and open it as the workspace.

```
[Terminal verification]
antigravity --version
node --version
npm --version
git --version
```

2. Create React with TypeScript using Vite

```
[Terminal]
npm create vite@latest unimart-frontend -- --template react-ts
cd unimart-frontend
npm install
npm run dev
```

Open the local URL displayed by Vite. Stop the server with Ctrl+C. Vite requires a supported Node version; use Node 22.12+ for a consistent class environment.

Tool choice: Create React App is deprecated. This guide uses the official React-supported Vite approach for a client-side learning project.

3. Clean the starter project

6. Remove unused demo assets and starter CSS rules.
7. Keep src/main.tsx, src/App.tsx, src/index.css, vite.config.ts and TypeScript configuration files.
8. Run npm run build before continuing. Fix TypeScript errors now rather than carrying them into later labs.

4. Recommended feature-first folder structure

```
[Folder tree]
src/
  app/
    store.ts
    hooks.ts
  assets/
  components/
    common/
    feedback/
  features/
    auth/
      authSlice.ts
      authTypes.ts
      components/
    listings/
      listingsApi.ts
      listingTypes.ts
      components/
      pages/
    reviews/
      reviewsApi.ts
      reviewTypes.ts
      components/
  layouts/
  routes/
    AppRouter.tsx
    ProtectedRoute.tsx
  services/
    baseApi.ts
  theme/
    theme.ts
  utils/
  App.tsx
  main.tsx
  index.css
```

Feature-first organization keeps catalog and review code cohesive and makes future extraction easier. app contains global configuration; features contain domain code; components contains genuinely reusable UI only.

5. Install Redux Toolkit and React Redux

```
[Terminal]
npm install @reduxjs/toolkit react-redux
```

```
[TypeScript]
// src/app/store.ts
import { configureStore } from '@reduxjs/toolkit';
import authReducer from '../features/auth/authSlice';

export const store = configureStore({
  reducer: {
    auth: authReducer,
  },
  devTools: import.meta.env.DEV,
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

```
[TypeScript]
// src/app/hooks.ts
import { useDispatch, useSelector } from 'react-redux';
import type { TypedUseSelectorHook } from 'react-redux';
import type { AppDispatch, RootState } from './store';

export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

```
[TypeScript]
// src/main.tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { store } from './app/store';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </StrictMode>,
);
```

6. Install Material UI

```
[Terminal]
npm install @mui/material @mui/icons-material @emotion/react @emotion/styled
npm install @fontsource/roboto
```

```
[TypeScript]
// src/theme/theme.ts
import { createTheme } from '@mui/material/styles';

export const theme = createTheme({
  palette: {
    mode: 'light',
    primary: { main: '#1f4e78' },
    secondary: { main: '#2e75b6' },
  },
  shape: { borderRadius: 8 },
});
```

```
[TSX]
// Wrap the Provider content in main.tsx
<ThemeProvider theme={theme}>
  <CssBaseline />
  <App />
</ThemeProvider>
```

7. Install Tailwind CSS v4 with the Vite plugin

```
[Terminal]
npm install tailwindcss @tailwindcss/vite
```

```
[TypeScript]
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: { port: 5173 },
});
```

```
[src/index.css]
@import "tailwindcss";

html, body, #root { min-height: 100%; }
```

Styling rule: Use Material UI for accessible interactive components and theme tokens. Use Tailwind mainly for layout, responsive spacing and simple utility styling. Avoid applying competing MUI sx props and Tailwind classes to the same CSS property.

8. Install routing and form utilities

```
[Terminal]
npm install react-router-dom react-hook-form zod @hookform/resolvers
```

```
[TSX]
// src/routes/AppRouter.tsx
import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const router = createBrowserRouter([
  { path: '/', lazy: () => import('../features/listings/pages/ListingsPage') },
  { path: '/listings/:id', lazy: () => import('../features/listings/pages/ListingDetailsPage') },
  { path: '/login', lazy: () => import('../features/auth/pages/LoginPage') },
]);

export default function AppRouter() {
  return <RouterProvider router={router} />;
}
```

9. Frontend environment variables

```
[/.env.local]
VITE_API_BASE_URL=http://localhost:8080/api/v1
```

```
[/.env.example]
VITE_API_BASE_URL=http://localhost:8080/api/v1
```

Only variables prefixed with VITE_ are exposed to client code. Never place passwords, JWT signing secrets, database credentials or private API keys in a Vite environment file; browser users can inspect bundled values.

10. Quality scripts

```
[Terminal]
npm run dev
npm run build
npm run lint
npm audit
npm outdated
```

11. Common errors

Problem	Likely cause	Fix
npm create vite fails	Unsupported Node version or proxy issue	Install Node 22.12+; verify npm registry/proxy; reopen terminal.
Blank page	Runtime error or wrong root element	Open browser Console; verify index.html contains id="root".
Tailwind classes do nothing	Vite plugin/import missing	Add tailwindcss() to vite.config.ts and @import "tailwindcss".
MUI font/icons missing	Packages/imports not installed	Install @fontsource/roboto and @mui/icons-material; restart dev server.
Redux state not visible	Provider missing	Wrap App in Provider and install Redux DevTools extension later.
Type errors after package updates	Lock-file mismatch	Delete node_modules only as last resort; run npm ci from a clean lock file.

Completion checklist

☐ Antigravity opens the UniMart workspace and integrated terminal.

☐ React TypeScript project starts on localhost and builds successfully.

☐ Feature-first folders are created.

☐ Redux store is connected to Provider.

☐ MUI theme and Tailwind utilities both render correctly.

☐ .env.local is ignored and .env.example is committed.